

CSA1711 ARTIFICIAL INTELLIGENCE /CO5-AT3

ANNEXURE A

Technical Presentation Content Pack

Intelligent Agent Architectures for Real-World AI Systems

Covers all five topics:

1. Reactive Agent-Based Traffic Control System
2. Goal-Based Agent for Healthcare Diagnosis
3. Utility-Based Agent for E-Commerce Recommendations
4. Learning Agent for Personalized Education System
5. Multi-Agent System for Disaster Management

Prepared as source content for slide-deck development and viva/presentation delivery.

Table of Contents

Table of Contents	2
How to Use This Document	4
1. Reactive Agent-Based Traffic Control System	5
1.1 Problem Identification	5
1.2 Agent Type and PEAS Description	5
1.3 Reactive Agent Architecture	5
1.4 Streaming Data Processing and Control Logic	5
1.5 Tools and Technologies to Cite	6
1.6 Suggested 10-Slide Outline	6
2. Goal-Based Agent for Healthcare Diagnosis	7
2.1 Problem Identification	7
2.2 Agent Type and PEAS Description	7
2.3 Goal-Based Reasoning Architecture	7
2.4 Decision-Making Logic and Data Pipeline	7
2.5 Tools and Technologies to Cite	8
2.6 Suggested 10-Slide Outline	8
3. Utility-Based Agent for E-Commerce Recommendations	9
3.1 Problem Identification	9
3.2 Agent Type and PEAS Description	9
3.3 Utility Function Design	9
3.4 Large-Scale Data Processing Pipeline	10
3.5 Tools and Technologies to Cite	10
3.6 Suggested 10-Slide Outline	10
4. Learning Agent for Personalized Education System	11
4.1 Problem Identification	11
4.2 Agent Type and the Four Components of a Learning Agent	11
4.3 PEAS Description	11
4.4 Adaptive Algorithm and Feedback Loop	12
4.5 Distributed Data Processing and Tools	12
4.6 Suggested 10-Slide Outline	12
5. Multi-Agent System for Disaster Management	14
5.1 Problem Identification	14
5.2 Agent Roles in the System	14
5.3 PEAS Description (System-Level)	14
5.4 Coordination Strategies and Communication Protocols	15

5.5 Real-Time Data Sharing and Decision-Making Pipeline.....	15
5.6 Tools and Technologies to Cite.....	15
5.7 Suggested 10-Slide Outline.....	16
Rubric Alignment Summary.....	17
Closing Notes.....	17

How to Use This Document

This document provides complete, presentation-ready technical content for all five topics in Annexure A. For each topic it covers:

- The engineering problem being solved and why it matters
- The agent type, its PEAS (Performance, Environment, Actuators, Sensors) description, and architecture diagram description
- The core decision-making / control algorithm, explained step by step
- The data pipeline and specific tools, frameworks, and technologies to cite
- A ready-to-use slide-by-slide outline (10 slides) for building the PPT
- Delivery notes aligned to the Communication and Professionalism rubric criteria

A rubric-alignment table is included at the end explaining exactly how this content earns marks under each weighted criterion.

1. Reactive Agent-Based Traffic Control System

1.1 Problem Identification

Urban traffic congestion arises because fixed-timer traffic signals cannot respond to real-time variation in vehicle density, pedestrian flow, or emergency vehicle movement. A signal cycle tuned for peak-hour traffic wastes green time on empty approaches during off-peak hours, while a cycle tuned for average traffic causes queue build-up during surges. The engineering problem is therefore: how can a signal controller make sub-second decisions using only the traffic conditions it can currently sense, without waiting for a central planner, so that average vehicle wait time and queue length are minimized at each intersection?

A Reactive (simple-reflex) Agent is the correct fit because the problem is local, time-critical, and does not require reasoning about future states or internal memory of history — the correct action can be computed directly from the current percept using condition-action rules.

1.2 Agent Type and PEAS Description

Element	Description	Examples in this System
Performance	Maximize traffic throughput; minimize average wait time, queue length, and emergency-vehicle delay.	Vehicles cleared per minute; average stop time per lane.
Environment	A single road intersection with multiple approach lanes, pedestrians, and connected neighboring signals.	4-way junction with induction loops, cameras, and pedestrian call buttons.
Actuators	The physical outputs the agent controls.	Traffic light phase (red/amber/green), pedestrian signal, variable message sign.
Sensors	Real-time inputs from the environment.	Inductive loop sensors, CCTV/computer-vision vehicle counts, RFID for emergency vehicles, pedestrian buttons.

1.3 Reactive Agent Architecture

A reactive agent maps percepts directly to actions through condition-action (if-then) rules, with no internal model of the world beyond the current sensor reading:

- Percept layer: streaming sensor data (loop detectors, camera vehicle counts, queue-length estimates) sampled every 1–2 seconds
- Rule/condition-action layer: a rule base such as ‘IF queue-length(North) > threshold AND phase ≠ North-green THEN switch-to(North-green)’
- Actuator layer: signal phase controller that executes the chosen action immediately

Because the agent has no memory of past states, robustness comes from well-designed rule priorities (e.g., emergency-vehicle preemption > pedestrian request > longest queue > default cycle) rather than planning.

1.4 Streaming Data Processing and Control Logic

The core control loop runs continuously on live data rather than batch data:

- **Ingestion:** Sensor events (loop triggers, camera frame counts, GPS pings from connected vehicles) are pushed onto a streaming message bus
- **Windowed aggregation:** A sliding time window (e.g., last 10 seconds) computes per-lane vehicle density and queue length
- **Rule evaluation:** The current aggregated state is evaluated against the condition-action rule set at every window tick
- **Conflict resolution:** A priority scheme resolves cases where multiple lanes qualify for green simultaneously (e.g., emergency preemption overrides all)
- **Actuation:** The selected phase change is sent to the signal controller with minimum green/amber safety timers enforced

This is essentially an event-driven, low-latency control loop — the defining characteristic of a reactive agent — rather than a plan-then-execute cycle.

1.5 Tools and Technologies to Cite

Layer	Recommended Tools	Purpose
Sensing	Inductive loop detectors, IP CCTV + YOLO/OpenCV vehicle detection, RFID emergency-vehicle tags	Real-time vehicle counting and classification
Streaming ingestion	Apache Kafka / MQTT broker	Low-latency transport of sensor events from intersections to the controller
Stream processing	Apache Flink or Spark Structured Streaming	Windowed aggregation of queue length and density
Rule engine	Drools / lightweight custom rule evaluator	Condition-action rule execution for phase decisions
Visualization	Grafana / Power BI dashboard	Live intersection dashboard for the presentation demo

1.6 Suggested 10-Slide Outline

- **Slide 1 — Title & Problem Statement:** Congestion cost, why fixed-timer signals fail.
- **Slide 2 — Why a Reactive Agent:** Contrast reactive vs. deliberative agents for this problem.
- **Slide 3 — PEAS Description:** Use the PEAS table above.
- **Slide 4 — System Architecture Diagram:** Sensors → stream bus → rule engine → signal actuator.
- **Slide 5 — Condition-Action Rule Set:** Show 4–5 example rules with priorities.
- **Slide 6 — Streaming Pipeline:** Kafka/Flink windowed aggregation walkthrough.
- **Slide 7 — Emergency Vehicle Preemption Case:** Worked example of rule conflict resolution.
- **Slide 8 — Live/Simulated Dashboard:** Grafana mock-up showing queue lengths per lane.
- **Slide 9 — Results / Expected Impact:** Projected % reduction in average wait time.
- **Slide 10 — Limitations & Future Work:** No memory of history; upgrade path to model-based agent.

2. Goal-Based Agent for Healthcare Diagnosis

2.1 Problem Identification

Clinical diagnosis requires combining heterogeneous patient data (symptoms, vitals, lab results, imaging, history) to reach a specific goal state — a confirmed diagnosis and an optimal treatment plan — while considering multiple plausible hypotheses and future consequences of each action (e.g., ordering a test, prescribing a drug). A purely reflexive system cannot handle this because the correct next action depends on what the ultimate diagnostic goal requires, not just the current percept. The engineering problem is: how do we design an agent that searches over possible diagnostic and treatment paths, evaluates whether each path leads toward the goal of an accurate, safe diagnosis, and selects the best next action accordingly?

A Goal-Based Agent is appropriate because it maintains explicit goals (confirm/rule out conditions, achieve safe treatment) and reasons about which sequence of actions will achieve them — essential when actions have delayed or uncertain effects, as in medicine.

2.2 Agent Type and PEAS Description

Element	Description	Examples in this System
Performance	Achieve an accurate diagnosis and safe, effective treatment while minimizing risk, cost, and time-to-treatment.	Diagnostic accuracy rate, time-to-diagnosis, avoidance of contraindicated treatments.
Environment	The patient's evolving clinical state, medical knowledge base, and hospital information systems.	EHR records, lab systems, imaging archives, clinician inputs.
Actuators	Actions available to move toward the goal.	Recommend a test, flag a differential diagnosis, suggest a treatment pathway, alert a clinician.
Sensors	Structured and unstructured patient data.	Vitals, symptom checklists, lab values, radiology reports, patient history.

2.3 Goal-Based Reasoning Architecture

Unlike a reactive agent, a goal-based agent maintains an internal model of the world (the patient's state) and searches over possible action sequences to determine which leads to the goal:

- Goal formulation: define the target state, e.g., ‘differential diagnosis narrowed to a single condition with confidence above threshold’
- State representation: current belief state over possible diagnoses, built from all evidence gathered so far
- Search / inference: evaluate candidate next actions (e.g., order test A vs. test B) by predicted information gain toward the goal
- Goal test: check whether the belief state satisfies the diagnostic confidence goal; if not, select the next best action and repeat

This can be implemented as a Bayesian belief-updating loop combined with a decision-tree or search-based planner over possible next tests/treatments.

2.4 Decision-Making Logic and Data Pipeline

- **Data ingestion:** Patient data streamed/pulled from EHR via HL7 FHIR APIs into a unified patient record
- **Feature extraction:** NLP pipeline extracts structured features from clinical notes; vitals and labs normalized
- **Hypothesis generation:** A classification/probabilistic model (e.g., Bayesian network or gradient-boosted classifier) proposes candidate diagnoses with confidence scores
- **Goal-directed action selection:** The agent computes, for each candidate next test, its expected effect on diagnostic confidence (expected information gain) and selects the action maximizing progress toward the goal
- **Treatment pathway selection:** Once the diagnosis goal is met, a clinical-pathway/guideline engine selects the treatment plan consistent with contraindications and patient-specific constraints
- **Human-in-the-loop:** All recommendations are surfaced to a clinician for confirmation — the agent supports, not replaces, medical judgment

2.5 Tools and Technologies to Cite

Layer	Recommended Tools	Purpose
Data integration	HL7 FHIR APIs, EHR connectors (Epic/Cerner interfaces)	Standardized patient data ingestion
NLP	spaCy / clinical BERT (Bio_ClinicalBERT)	Extracting symptoms and findings from clinical notes
Modeling	Bayesian networks (pgmpy), scikit-learn / XGBoost classifiers	Diagnostic hypothesis scoring
Knowledge base	SNOMED CT, ICD-10 ontologies, clinical guideline repositories	Grounding diagnoses and treatment pathways in standards
Data pipeline	Apache Airflow	Orchestrating ingestion, feature extraction, and scoring jobs
Visualization	Dashboards showing differential diagnosis ranking and confidence	Clinician-facing decision support UI

2.6 Suggested 10-Slide Outline

- **Slide 1 — Title & Problem Statement:** Complexity of multi-hypothesis clinical diagnosis.
- **Slide 2 — Why a Goal-Based Agent:** Need for goal-directed reasoning vs. reflex response.
- **Slide 3 — PEAS Description:** Use the PEAS table above.
- **Slide 4 — System Architecture Diagram:** EHR → feature extraction → belief state → goal test → action.
- **Slide 5 — Belief-Updating / Diagnostic Model:** Bayesian network or classifier walkthrough.
- **Slide 6 — Goal-Directed Test Selection:** Worked example: choosing next test by information gain.
- **Slide 7 — Treatment Pathway Selection:** Guideline-engine example for a confirmed diagnosis.
- **Slide 8 — Data Pipeline:** FHIR ingestion → Airflow orchestration → model scoring.
- **Slide 9 — Safety & Human Oversight:** Human-in-the-loop confirmation, contraindication checks.
- **Slide 10 — Results & Ethical Considerations:** Accuracy gains, bias/fairness, data privacy (HIPAA).

3. Utility-Based Agent for E-Commerce Recommendations

3.1 Problem Identification

In e-commerce, multiple candidate products can each satisfy a user's stated goal (e.g., 'buy a laptop'), but goals alone cannot rank them — the system must choose the option that best balances competing factors such as price, relevance, brand affinity, delivery time, and predicted satisfaction. A simple goal-based agent can only say whether a state satisfies the goal, not how good one satisfying state is relative to another. The engineering problem is: how do we rank a large candidate set of products by a single, well-calibrated measure of expected user satisfaction, computed at scale and in real time?

A Utility-Based Agent is the correct fit because it assigns a numeric utility to each possible outcome and selects the action (recommendation) that maximizes expected utility, naturally handling trade-offs between multiple, sometimes conflicting, objectives.

3.2 Agent Type and PEAS Description

Element	Description	Examples in this System
Performance	Maximize expected user utility — a weighted combination of relevance, satisfaction, conversion likelihood, and long-term engagement.	Click-through rate, conversion rate, return-customer rate.
Environment	The online store: catalog, user browsing/purchase history, real-time session behavior, inventory and pricing.	Product catalog, session clickstream, cart events.
Actuators	Actions the agent can take.	Display a ranked recommendation list, personalize a banner, send a targeted offer.
Sensors	Signals about the user and context.	Browsing history, past purchases, search queries, device/location, real-time clicks.

3.3 Utility Function Design

The utility function combines multiple weighted signals into a single score used to rank candidate products for a given user and context:

- $U(\text{product}) = w_1 \cdot \text{Relevance} + w_2 \cdot \text{PredictedRating} + w_3 \cdot \text{PriceFit} + w_4 \cdot \text{Availability/DeliverySpeed} + w_5 \cdot \text{ProfitMargin} - w_6 \cdot \text{Risk}(\text{returns/complaints})$
- Relevance is typically produced by a content-based or collaborative-filtering similarity score
- PredictedRating comes from a learned model (matrix factorization or deep recommender) estimating how much the user would like the item
- Weights ($w_1 \dots w_6$) are tuned via offline evaluation (e.g., NDCG) and online A/B testing to reflect business and user-satisfaction priorities

At serving time, the agent scores all candidate products for the current user/context and returns the top-N by utility, rather than merely filtering products that satisfy a goal.

3.4 Large-Scale Data Processing Pipeline

- **Candidate generation:** A fast approximate-nearest-neighbor retrieval step narrows millions of products to a few hundred candidates per user
- **Feature computation:** Batch (historical) and real-time (session) features are computed and joined — e.g., via a feature store
- **Utility scoring:** A ranking model (e.g., gradient-boosted trees or a two-tower neural network) computes the utility score per candidate
- **Re-ranking:** Business rules (diversity, inventory constraints, promotions) adjust the raw utility ranking before display
- **Feedback loop:** Clicks/purchases are logged and fed back to retrain the utility model, closing the loop

3.5 Tools and Technologies to Cite

Layer	Recommended Tools	Purpose
Candidate retrieval	FAISS / ScaNN (approximate nearest neighbor)	Fast similarity search over millions of products
Big data processing	Apache Spark / PySpark	Batch feature engineering over historical interactions
Feature store	Feast	Serving consistent features online and offline
Ranking model	XGBoost / LightGBM, or a two-tower neural recommender (TensorFlow Recommenders)	Utility score prediction
Real-time serving	Redis + a low-latency model server (TensorFlow Serving / Triton)	Sub-100ms recommendation responses
Experimentation	A/B testing platform (e.g., Optimizely-style setup)	Validating utility weight changes against real user behavior

3.6 Suggested 10-Slide Outline

- **Slide 1 — Title & Problem Statement:** Why relevance alone isn't enough; need for trade-off ranking.
- **Slide 2 — Why a Utility-Based Agent:** Contrast with goal-based agents that only test satisfaction.
- **Slide 3 — PEAS Description:** Use the PEAS table above.
- **Slide 4 — Utility Function Design:** Show the weighted formula and explain each term.
- **Slide 5 — System Architecture Diagram:** Candidate generation → feature store → ranking → re-ranking.
- **Slide 6 — Ranking Model:** Two-tower / gradient-boosted model overview.
- **Slide 7 — Large-Scale Pipeline:** PySpark batch features + Feast + real-time serving.
- **Slide 8 — Worked Example:** Rank 3 candidate laptops by utility score for a sample user.
- **Slide 9 — A/B Testing & Feedback Loop:** How weights are tuned from live results.
- **Slide 10 — Results & Business Impact:** Projected uplift in conversion / engagement.

4. Learning Agent for Personalized Education System

4.1 Problem Identification

A static curriculum delivers the same content and difficulty to every learner regardless of their pace, misconceptions, or mastery level, leading to disengagement for advanced learners and frustration for struggling ones. The engineering problem is: how can a tutoring system continuously observe a learner's performance, infer their current mastery level, and adapt content difficulty and sequencing over time — improving its own model of the learner as more data arrives, rather than using a fixed policy?

A Learning Agent is the correct fit because the problem explicitly requires performance to improve with experience: the agent must update its internal model (of both the learner and the effectiveness of its own teaching actions) based on feedback, which reactive, goal-based, or utility-based agents alone do not do.

4.2 Agent Type and the Four Components of a Learning Agent

Component	Role	Example in This System
Performance Element	Selects the actual teaching action given the current learner model	Chooses next question/content item and difficulty level
Critic	Evaluates outcomes against a performance standard and produces feedback	Compares predicted vs. actual quiz score to judge if difficulty was well-calibrated
Learning Element	Updates the performance element based on critic feedback	Retrains/updates the learner-mastery model (e.g., Bayesian Knowledge Tracing parameters)
Problem Generator	Suggests exploratory actions to gather informative new data	Occasionally presents a slightly harder/easier item to probe true mastery boundary

4.3 PEAS Description

Element	Description	Examples in this System
Performance	Maximize long-term learning gains and engagement while keeping content in the learner's zone of proximal development.	Mastery growth rate, completion rate, engagement/retention.
Environment	The learner, the content repository, and the learning-management platform.	Course content graph, quiz bank, learner interaction logs.
Actuators	Actions the agent can take.	Present a content item, adjust difficulty, recommend a review module, trigger a hint.
Sensors	Signals observed from the learner.	Answer correctness, response time, hint usage, engagement/click patterns.

4.4 Adaptive Algorithm and Feedback Loop

- **Mastery estimation:** A model such as Bayesian Knowledge Tracing or a deep knowledge-tracing (recurrent) model maintains a probability of mastery per skill/topic for each learner
- **Action selection (performance element):** Given current mastery estimates, an item-selection policy (e.g., contextual bandit or item-response-theory-based selector) picks the next content item at an appropriate difficulty
- **Critic evaluation:** After the learner responds, the critic compares predicted correctness probability to the actual outcome to measure prediction/calibration error
- **Model update (learning element):** Mastery model parameters are updated online (e.g., via gradient updates or Bayesian posterior update) using the observed response
- **Exploration (problem generator):** A small exploration rate occasionally serves items outside the current estimated comfort zone to refine the model and avoid premature convergence

4.5 Distributed Data Processing and Tools

Layer	Recommended Tools	Purpose
Interaction logging	Kafka / event logging SDK in the LMS	Capturing every answer, hint use, and time-on-task event
Distributed processing	PySpark (Spark MLlib)	Batch retraining of mastery models across large learner populations
Online model updates	Streaming job (Spark Structured Streaming or Flink)	Near-real-time mastery model updates per learner session
Model type	Bayesian Knowledge Tracing / Deep Knowledge Tracing (LSTM-based)	Modeling per-skill mastery probability over time
Content adaptation	Item Response Theory (IRT) engine or contextual bandit (e.g., Vowpal Wabbit)	Choosing next-best content item
Dashboards	Learner/teacher analytics dashboard	Visualizing mastery growth and engagement trends

4.6 Suggested 10-Slide Outline

- **Slide 1 — Title & Problem Statement:** Limits of one-size-fits-all curricula.
- **Slide 2 — Why a Learning Agent:** Needs to improve its own policy from feedback over time.
- **Slide 3 — Four Components of a Learning Agent:** Performance element, critic, learning element, problem generator.
- **Slide 4 — PEAS Description:** Use the PEAS table above.
- **Slide 5 — System Architecture Diagram:** Learner interactions → PySpark pipeline → mastery model → content adaptation.
- **Slide 6 — Mastery Model:** Bayesian/Deep Knowledge Tracing explained simply.
- **Slide 7 — Adaptive Item Selection:** IRT / contextual bandit worked example.
- **Slide 8 — Feedback Loop in Action:** Walk through one learner session end-to-end.

- **Slide 9 — Distributed Processing at Scale:** PySpark batch retraining across thousands of learners.
- **Slide 10 — Results & Future Work:** Learning gains, cold-start problem, fairness across learner groups.

5. Multi-Agent System for Disaster Management

5.1 Problem Identification

Disaster response (e.g., flood, earthquake, wildfire) requires simultaneously sensing a wide geographic area, interpreting that data into actionable decisions, and coordinating communication and resource deployment — a scope and speed no single centralized agent can handle reliably, especially when network connectivity is partial or intermittent. The engineering problem is: how do we design a system of specialized, distributed agents that can independently sense, decide, and communicate, while still producing coherent, coordinated, real-time disaster response actions across the affected region?

A Multi-Agent System (MAS) is the correct fit because it decomposes the problem into specialized roles (sensing, decision, communication) that operate concurrently and coordinate through message passing, providing scalability and fault tolerance that a single monolithic agent cannot.

5.2 Agent Roles in the System

Agent Type	Responsibility	Example Behavior
Sensor Agents	Continuously monitor local conditions and detect anomalies	IoT/satellite/drone agent detects rising water level or seismic activity and raises an alert
Decision Agents	Aggregate alerts, assess severity, and decide on response actions	Regional agent fuses multiple sensor alerts, classifies severity, and selects an evacuation/resource-allocation plan
Communication Agents	Relay information between sensor, decision, and human responder layers, including under degraded networks	Mesh-network agent routes alerts to nearest operational base station or satellite uplink
Resource/Coordination Agents (optional)	Allocate scarce resources (ambulances, shelters, rescue teams) across competing demands	Auction- or negotiation-based allocation of rescue units to highest-priority zones

5.3 PEAS Description (System-Level)

Element	Description	Examples in this System
Performance	Minimize response time, maximize lives saved and resource-utilization efficiency, maintain accurate situational awareness.	Time-to-alert, time-to-resource-arrival, coverage completeness.
Environment	A large, dynamic, partially observable geographic region with unreliable communication links.	Flood plain / earthquake zone with sensor networks, drones, responder units.

Element	Description	Examples in this System
Actuators	Actions distributed across the agent types.	Raise alert, issue evacuation order, dispatch resource, relay message.
Sensors	Distributed, heterogeneous sensing.	Water-level sensors, seismographs, satellite imagery, social-media/crowd reports, drone video.

5.4 Coordination Strategies and Communication Protocols

- Publish-subscribe messaging: sensor agents publish readings/alerts to topics; decision agents subscribe to relevant regional topics — decouples agents and supports scale
- Contract-net protocol: decision/coordination agents ‘announce’ a task (e.g., dispatch a rescue team) and resource agents ‘bid’ based on availability and distance, with the best bid awarded the task
- Belief consensus: when multiple sensor agents report conflicting readings for the same area, decision agents apply a voting/weighted-confidence fusion rule before acting
- Fault tolerance: communication agents use mesh networking / store-and-forward relay so the system degrades gracefully rather than failing completely if part of the network goes down
- Hierarchical + peer coordination: local decision agents act autonomously for fast local response, while a regional coordination layer resolves cross-zone resource conflicts

5.5 Real-Time Data Sharing and Decision-Making Pipeline

- **Sensing layer:** Heterogeneous sensor agents (IoT, drones, satellite feeds, crowd-sourced reports) stream raw readings continuously
- **Ingestion & fusion:** A distributed message broker collects and time-aligns readings from all sensor agents across the region
- **Situational assessment:** Decision agents run anomaly detection / severity classification models on fused data to identify emerging incidents
- **Action selection:** Decision agents choose response actions (alerts, evacuation, resource requests) and negotiate resource allocation with coordination agents via contract-net or auction protocols
- **Dissemination:** Communication agents relay decisions to responders and the public through redundant channels (SMS, radio mesh, apps)

5.6 Tools and Technologies to Cite

Layer	Recommended Tools	Purpose
Multi-agent framework	JADE (Java Agent DEvelopment) or SPADE (Python)	Implementing autonomous agents and FIPA-style communication
Messaging/coordination	MQTT / Apache Kafka with pub-sub topics per region	Distributed, low-latency alert and data sharing
Sensing	IoT sensor networks, drone imagery, satellite feeds (e.g., open satellite APIs)	Wide-area environmental monitoring

Layer	Recommended Tools	Purpose
Situational ML models	CNN-based image anomaly detection, time-series anomaly detection (e.g., Prophet, LSTM)	Detecting flood/fire/seismic anomalies from sensor streams
Geospatial visualization	GIS dashboard (e.g., Leaflet/Mapbox-based map)	Live map of alerts, resources, and evacuation routes for the demo
Resilient networking	Mesh networking protocols (e.g., LoRa, ad-hoc Wi-Fi mesh)	Maintaining communication when infrastructure is damaged

5.7 Suggested 10-Slide Outline

- **Slide 1 — Title & Problem Statement:** Scale/speed challenges of disaster response.
- **Slide 2 — Why a Multi-Agent System:** Decomposition into sensor/decision/communication roles.
- **Slide 3 — Agent Roles Table:** Use the roles table above.
- **Slide 4 — System-Level PEAS:** Use the PEAS table above.
- **Slide 5 — Architecture Diagram:** Distributed sensors → message bus → decision agents → comms agents → responders.
- **Slide 6 — Coordination Protocol:** Contract-net worked example for rescue-team dispatch.
- **Slide 7 — Fault-Tolerant Communication:** Mesh networking / store-and-forward under network damage.
- **Slide 8 — Live Dashboard Concept:** GIS map mock-up with live alerts and resource positions.
- **Slide 9 — Case Walkthrough:** End-to-end trace: flood sensor alert → evacuation dispatch.
- **Slide 10 — Results, Ethics & Future Work:** Response-time gains, data-sharing privacy, cross-agency interoperability.

Rubric Alignment Summary

The table below maps this content pack directly to the weighted grading rubric, so each section can be checked off before the final presentation is delivered.

Criterion (Weight)	How This Content Pack Addresses It	Presenter Checklist
Technical Content Accuracy (30%)	Correct AI-agent terminology (PEAS, reactive vs. deliberative vs. utility vs. learning vs. multi-agent), accurate architecture and algorithm descriptions for each topic.	Rehearse explaining the PEAS table and core algorithm in your own words without reading slides.
Problem Identification (25%)	Each topic opens with a clearly framed real-world problem and a justification for why that specific agent type is the right engineering choice.	Be ready to answer: 'why not a simpler agent type?' for your topic.
Use of Tools (20%)	Named, appropriate tools per layer (sensing, streaming, ML, visualization) plus a described end-to-end data pipeline.	Prepare one visual (architecture diagram or dashboard mock-up) per topic.
Communication (15%)	A ready 10-slide outline with one clear idea per slide, in a logical narrative order.	Time your rehearsal; keep each slide to ~1–1.5 minutes.
Professionalism (10%)	Ethics/safety notes included per topic (human-in-the-loop for healthcare, data privacy, fairness, cross-agency coordination).	Assign team roles for Q&A and mention ethical considerations even if not directly asked.

Closing Notes

This document is source content, not a slide deck — use the 10-slide outlines to build the actual PPT for each topic, and pull the tables/diagrams described here directly onto slides as visuals. Each topic section is self-contained, so a team member can be assigned one topic and present independently while staying consistent with the overall PEAS/architecture/tools/outline structure used throughout.